

Automated Code Audit — Executive Summary

Prepared for: Engineering Leadership | File reviewed: process_user_login (login module) | Date: September 2026

Overview

As part of our ongoing effort to catch security and quality issues before they reach production, we ran an automated AI-powered audit on a sample login module. The scan identified **multiple critical security weaknesses** that should be addressed before this code is deployed. None of the issues found are difficult to fix, but leaving them unaddressed creates real risk to customer data and account security.

Risk Snapshot

Severity	Issues Found	What This Means for the Business
Critical	2	Immediate risk — customer accounts and credentials could be compromised.
High	1	Serious weakness that should be fixed before the next release.
Medium / Low	2	Lower urgency, but worth cleaning up for long-term stability.

Top Findings, In Plain Language

- **A hidden "back door" into the database.** The way this code builds its database queries would let an attacker type specially-crafted text into a login box and potentially read, change, or delete data they shouldn't have access to. This is one of the most common ways real companies get breached.
- **A secret access key was left directly in the code.** If this code were ever shared, backed up, or pushed to a public repository, anyone who saw it could use that key to access protected systems.
- **Passwords are handled in an unsafe way.** Best practice is to never store or compare passwords in plain, readable form — they should be scrambled ("hashed") so that even we can't see them.

Recommendation

We recommend these issues be fixed **before** this module goes live. The engineering team has already reviewed the technical fixes required, and none of them are expected to take more than a few hours to implement and verify. We also recommend running this same automated audit on every future code change, so issues like these are caught automatically going forward rather than found after the fact.